

CSS Animation-Fill-Mode Property

Complete Guide: Easy English Explanation

Contents

1	Introduction	2
1.1	What is Animation-Fill-Mode?	2
1.2	Why Use Animation-Fill-Mode?	2
1.3	The Problem It Solves	2
2	Core Concept: Fill-Mode Values	4
2.1	Value 1: none (Default)	4
2.1.1	What It Does	4
2.1.2	Timeline	4
2.1.3	Code Example	4
2.1.4	When to Use none	4
2.2	Value 2: forwards	5
2.2.1	What It Does	5
2.2.2	Timeline	5
2.2.3	Code Example	5
2.2.4	When to Use forwards	5
2.3	Value 3: backwards	5
2.3.1	What It Does	5
2.3.2	Timeline	6
2.3.3	Code Example	6
2.3.4	When to Use backwards	6
2.4	Value 4: both	6
2.4.1	What It Does	6
2.4.2	Timeline	6
2.4.3	Code Example	7
2.4.4	When to Use both	7
3	Side-by-Side Comparison	8
3.1	All Four Values Compared	8
3.1.1	Scenario: 2-second fade-in with 1-second delay	8
3.2	Visual Timeline Comparison	9
4	Practical Examples	10
4.1	Example 1: Page Load Fade In	10
4.2	Example 2: Slide In with Delay	10
4.3	Example 3: Modal Entrance	11
4.4	Example 4: Bounce Animation (Repeating)	11

4.5	Example 5: Loading Spinner	12
4.6	Example 6: Button Hover Animation	12
4.7	Example 7: Sequential Staggered Load	13
4.8	Example 8: Overlay with Backwards	13
4.9	Example 9: Fade In on Page Load (Common Pattern)	14
4.10	Example 10: Hero Section Animation	14
5	Common Mistakes	15
5.1	Mistake 1: Forgetting forwards for One-Time Animations	15
5.2	Mistake 2: Using forwards for Repeating Animations	15
5.3	Mistake 3: Confused About Delay Behavior	16
5.4	Mistake 4: Using both When forwards is Enough	16
6	Decision Guide	18
6.1	How to Choose Fill-Mode	18
6.2	Quick Decision Table	18
7	Browser Support	19
7.1	Compatibility	19
7.2	Vendor Prefixes	19
8	Advanced Usage	20
8.1	Combining with JavaScript	20
8.2	Multiple Animations with Different Fill-Modes	20
9	Quick Reference	22
9.1	Fill-Mode Values Summary	22
9.2	Complete Checklist	22
9.3	Common Patterns	22
9.4	Related Animation Properties	23
9.5	Pro Tips	23

Chapter 1

Introduction

1.1 What is Animation-Fill-Mode?

The `animation-fill-mode` property controls what happens to an element **before the animation starts** and **after the animation ends**. It defines the "before" and "after" styles when the animation is not actively playing.

Think of fill-mode like a painting:

- **Before painting:** The canvas is blank (or has existing color)
- **During painting:** The artist paints a masterpiece
- **After painting:** The painting remains or the canvas returns to blank

Fill-mode controls what the canvas looks like before and after the painting!

1.2 Why Use Animation-Fill-Mode?

Animation-fill-mode is essential for:

- Keeping animations in their final state (don't jump back)
- Starting animations at specific positions (not the original)
- Creating smooth transitions between states
- Page load animations (fade in, slide in)
- Timed animations with delays
- One-time animations that stay completed
- Synchronized animations across multiple elements

1.3 The Problem It Solves

Without animation-fill-mode:

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.element {
  animation: fadeIn 1s;
  /* After animation ends, it JUMPS BACK to opacity: 0! */
  /* This looks broken and confusing */
}
```

With animation-fill-mode:

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.element {
  animation: fadeIn 1s forwards;
  /* After animation ends, STAYS at opacity: 1 */
  /* Looks smooth and intentional */
}
```

Chapter 2

Core Concept: Fill-Mode Values

2.1 Value 1: none (Default)

2.1.1 What It Does

The **none** value means no special styling before or after the animation. The element returns to its default CSS styles when the animation finishes. During animation delays, the element is in its normal state.

2.1.2 Timeline

0s	1s delay	2s starts	3s ends	4s after
-----	-----	-----	-----	
NORMAL	NORMAL	ANIMATING	ANIMATING	BACK TO NORMAL
STATE	STATE	(0%-100%)	(ends)	(jumps back)

2.1.3 Code Example

```
@keyframes slideIn {
  0% { transform: translateX(-100px); }
  100% { transform: translateX(0); }
}

.element {
  animation: slideIn 2s;
  animation-fill-mode: none; /* or just omit it */
  /* After animation ends: jumps back to translateX(-100px) */
}
```

Result: Element slides in, then jumps back to starting position.

2.1.4 When to Use none

- Hover effects that should reverse
- Repeating animations (like spinners)
- Effects where bouncing back looks natural

- Loading indicators

2.2 Value 2: forwards

2.2.1 What It Does

The **forwards** value keeps the final keyframe styles after the animation completes. The element stays in the 100

2.2.2 Timeline

0s	1s delay	2s starts	3s ends	4s after
-----	-----	-----	-----	
NORMAL	NORMAL	ANIMATING	ANIMATING	FINAL STATE
STATE	STATE	(0%-100%)	(ends)	(stays at 100%)

2.2.3 Code Example

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.element {
  animation: fadeIn 1s;
  animation-fill-mode: forwards;
  /* After animation ends: stays at opacity: 1 */
}
```

Result: Element fades in and stays visible.

2.2.4 When to Use forwards

- Page load animations (fade in, slide in)
- One-time animations that should stay completed
- Button reveal animations
- Loading screen transitions
- Most practical use case!

2.3 Value 3: backwards

2.3.1 What It Does

The **backwards** value applies the starting keyframe (0

2.3.2 Timeline

0s	1s delay	2s starts	3s ends	4s after
-----	-----	-----	-----	
NORMAL	FIRST	ANIMATING	ANIMATING	BACK TO
STATE	KEYFRAME	(0%-100%)	(ends)	NORMAL
	(0%)			STATE

2.3.3 Code Example

```
@keyframes slideIn {
  0% { transform: translateX(-200px); }
  100% { transform: translateX(0); }
}

.element {
  animation: slideIn 2s;
  animation-delay: 1s;
  animation-fill-mode: backwards;
}

/*
Timeline:
- 0s-1s: Element is at translateX(-200px) (starting position)
- 1s-3s: Animation plays (slides in)
- 3s+: Element returns to normal (translateX(0))
*/
```

Result: Element waits at starting position, slides in, then returns to normal.

2.3.4 When to Use backwards

- Delayed animations where you want to preview the start
- Sequential animations with delays
- Preparing element for animation visually
- Rarely used in practice

2.4 Value 4: both

2.4.1 What It Does

The **both** value combines **forwards** and **backwards**. Before the animation, it applies the first keyframe (0

2.4.2 Timeline

0s	1s delay	2s starts	3s ends	4s after
-----	-----	-----	-----	
NORMAL	FIRST	ANIMATING	ANIMATING	FINAL STATE
STATE	KEYFRAME	(0%-100%)	(ends)	(stays at 100%)
	(0%)			

2.4.3 Code Example

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.element {
  animation: fadeIn 1s;
  animation-delay: 0.5s;
  animation-fill-mode: both;
}

/*
Timeline:
- 0s-0.5s: Element is hidden (opacity: 0)
- 0.5s-1.5s: Animation plays (fades in)
- 1.5s+: Element stays visible (opacity: 1)
*/
```

Result: Element is hidden during delay, fades in, then stays visible.

2.4.4 When to Use **both**

- Delayed animations where preview state matters
- Complex animations with multiple stages
- Staggered animations across many elements
- When you need precise control over before/after

Chapter 3

Side-by-Side Comparison

3.1 All Four Values Compared

3.1.1 Scenario: 2-second fade-in with 1-second delay

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

/* none (default) */
.fade-none {
  animation: fadeIn 2s 1s none;
}

/* 0-1s: opacity normal (1)
   1-3s: fades from 0 to 1
   3s+: jumps back to 1 (normal) */

/* forwards */
.fade-forwards {
  animation: fadeIn 2s 1s forwards;
}

/* 0-1s: opacity normal (1)
   1-3s: fades from 0 to 1
   3s+: stays at opacity: 1 */

/* backwards */
.fade-backwards {
  animation: fadeIn 2s 1s backwards;
}

/* 0-1s: opacity 0 (first keyframe)
   1-3s: fades from 0 to 1
   3s+: jumps back to 1 (normal) */

/* both */
.fade-both {
  animation: fadeIn 2s 1s both;
}

/* 0-1s: opacity 0 (first keyframe)
```

```
1-3s: fades from 0 to 1  
3s+: stays at opacity: 1 */
```

3.2 Visual Timeline Comparison

Fill-Mode	Before Delay	During Delay	Animation	After
none	normal	normal	0% → 100%	normal
forwards	normal	normal	0% → 100%	100%
backwards	normal	0%	0% → 100%	normal
both	normal	0%	0% → 100%	100%

Chapter 4

Practical Examples

4.1 Example 1: Page Load Fade In

Element should fade in when page loads and stay visible:

```
@keyframes fadeIn {
  0% {
    opacity: 0;
  }
  100% {
    opacity: 1;
  }
}

.hero {
  animation: fadeIn 1s ease-in-out forwards;
  /* MUST use forwards! Otherwise it fades back out */
}
```

Without **forwards**: Element fades in, then disappears (bad!) With **forwards**: Element fades in and stays (good!)

4.2 Example 2: Slide In with Delay

Multiple elements slide in one after another:

```
@keyframes slideIn {
  0% {
    opacity: 0;
    transform: translateX(-50px);
  }
  100% {
    opacity: 1;
    transform: translateX(0);
  }
}

.item {
  animation: slideIn 0.5s ease-out forwards;
}
```

```
.item:nth-child(1) { animation-delay: 0s; }
.item:nth-child(2) { animation-delay: 0.1s; }
.item:nth-child(3) { animation-delay: 0.2s; }
.item:nth-child(4) { animation-delay: 0.3s; }

/* Each item slides in at different time and STAYS visible */
```

With **forwards**: Cascade of items sliding in (professional) Without **forwards**: Items pop back (looks broken)

4.3 Example 3: Modal Entrance

Modal should fade in and stay visible when triggered:

```
@keyframes modalFadeIn {
  0% {
    opacity: 0;
    transform: scale(0.9);
  }
  100% {
    opacity: 1;
    transform: scale(1);
  }
}

.modal {
  animation: modalFadeIn 0.3s ease-out forwards;
  /* Stays at final state (visible and full size) */
}

.modal.closing {
  /* When closing, reverse the effect */
  animation: none;
  opacity: 0;
  transform: scale(0.9);
}
```

4.4 Example 4: Bounce Animation (Repeating)

Bouncing element should return to start between bounces:

```
@keyframes bounce {
  0%, 100% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-30px);
  }
}

.ball {
```

```
animation: bounce 1s ease-in-out infinite none;
/* Use none because it repeats infinitely */
/* After one bounce, returns to start for next bounce */
}
```

With **none**: Ball bounces naturally, returns for next bounce With **forwards**: Ball would get stuck in final state (not ideal)

4.5 Example 5: Loading Spinner

Spinner should rotate continuously:

```
@keyframes spin {
  0% {
    transform: rotate(0deg);
  }
  100% {
    transform: rotate(360deg);
  }
}

.spinner {
  animation: spin 2s linear infinite none;
  /* Use none because continuous rotation needs to reset */
}
```

4.6 Example 6: Button Hover Animation

Button should have animation on hover that reverts when mouse leaves:

```
@keyframes buttonGrow {
  0% {
    transform: scale(1);
  }
  100% {
    transform: scale(1.05);
  }
}

.button {
  transition: transform 0.2s;
}

.button:hover {
  animation: buttonGrow 0.3s ease-out forwards;
}

/* When mouse leaves, grows back to normal */
```

4.7 Example 7: Sequential Staggered Load

Items appear one by one with delays:

```
@keyframes revealItem {
  0% {
    opacity: 0;
    transform: translateY(20px);
  }
  100% {
    opacity: 1;
    transform: translateY(0);
  }
}

.list-item {
  animation: revealItem 0.6s ease-out both;
  /* Use both: hidden during delay, stays visible after */
}

.list-item:nth-child(1) { animation-delay: 0s; }
.list-item:nth-child(2) { animation-delay: 0.1s; }
.list-item:nth-child(3) { animation-delay: 0.2s; }
.list-item:nth-child(4) { animation-delay: 0.3s; }

/*
With both:
- Item 2 is hidden at 0s (doesn't appear yet)
- Item 2 reveals at 0.1s (delay complete)
- Item 2 stays visible (forwards effect)

With forwards only:
- Item 2 visible at 0s (looks odd, no delay preview)
- Item 2 reveals at 0.1s
- Item 2 stays visible

With none:
- Item 2 jumps back when done (looks broken)
*/
```

4.8 Example 8: Overlay with Backwards

Overlay should show starting state during delay:

```
@keyframes overlaySlide {
  0% {
    transform: translateY(-100%);
    opacity: 0;
  }
  100% {
    transform: translateY(0);
    opacity: 1;
  }
}
```

```
}

.overlay {
  animation: overlaySlide 1s ease-out 2s backwards;
  /* 0-2s: Overlay positioned at top (off-screen)
     2-3s: Slides down into view
     3s+: Returns to normal position */
}
```

4.9 Example 9: Fade In on Page Load (Common Pattern)

```
@keyframes pageLoadFade {
  0% {
    opacity: 0;
  }
  100% {
    opacity: 1;
  }
}

body {
  animation: pageLoadFade 0.5s ease-in forwards;
  /* Page fades in smoothly on load */
}
```

4.10 Example 10: Hero Section Animation

```
@keyframes heroSlideIn {
  0% {
    opacity: 0;
    transform: translateY(30px);
  }
  100% {
    opacity: 1;
    transform: translateY(0);
  }
}

.hero-content {
  animation: heroSlideIn 0.8s ease-out forwards;
  /* Content slides up and fades in, stays visible */
}

.hero-button {
  animation: heroSlideIn 0.8s ease-out 0.2s forwards;
  /* Button follows with delay, also stays visible */
}
```


Chapter 5

Common Mistakes

5.1 Mistake 1: Forgetting forwards for One-Time Animations

Wrong:

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.element {
  animation: fadeIn 1s;
  /* Element fades in, then instantly disappears! */
}
```

Result: Looks broken and confusing.

Correct:

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.element {
  animation: fadeIn 1s forwards;
  /* Element fades in and STAYS visible */
}
```

Result: Smooth, professional appearance.

5.2 Mistake 2: Using forwards for Repeating Animations

Wrong:

```
@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

.spinner {
```

```
animation: spin 2s linear infinite forwards;
/* Spinner gets stuck after first rotation! */
}
```

Result: Spinner rotates once then stops.

Correct:

```
@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

.spinner {
  animation: spin 2s linear infinite;
  /* or: animation-fill-mode: none; */
  /* Spinner rotates continuously */
}
```

Result: Spinner loops smoothly.

5.3 Mistake 3: Confused About Delay Behavior

Common Confusion:

```
@keyframes slideIn {
  0% { transform: translateX(-100px); opacity: 0; }
  100% { transform: translateX(0); opacity: 1; }
}

.element {
  animation: slideIn 2s 1s forwards;
  /* Question: What does element look like during the 1s delay? */
}

/* With forwards: Still at normal position (not slid in yet)
   With backwards: Already at starting position (off-screen)
   With both: Already at starting position (off-screen)
*/
```

5.4 Mistake 4: Using both When forwards is Enough

Unnecessary:

```
@keyframes fadeIn {
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.element {
  animation: fadeIn 1s both;
  /* If no delay, both is same as forwards */
}
```

Better:

```
.element {  
  animation: fadeIn 1s forwards;  
  /* Cleaner and same result */  
}
```

Chapter 6

Decision Guide

6.1 How to Choose Fill-Mode

1. Does the animation repeat infinitely?
 - YES → Use **none** (default)
 - NO → Continue to step 2
2. Should the element keep its final state after animation?
 - YES → Use **forwards**
 - NO → Continue to step 3
3. Does the animation have a delay AND you want to see starting state during delay?
 - YES → Use **both** or **backwards**
 - NO → Use **none**

6.2 Quick Decision Table

Animation Type	Fill-Mode	Reason
Page load fade in	forwards	Stay visible
Loading spinner	none	Loop infinitely
Bounce effect	none	Reset between bounces
Button hover	forwards	Stay in hovered state
Staggered reveal	both	Preview during delay
Hover animation	none	Return to normal on leave
Modal entrance	forwards	Stay visible
Infinite rotation	none	Loop smoothly

Chapter 7

Browser Support

7.1 Compatibility

`animation-fill-mode` is supported in all modern browsers:

- Chrome 26+
- Firefox 16+
- Safari 9+
- Edge (all versions)
- Opera 15+
- iOS Safari 9+
- Chrome Mobile 26+

7.2 Vendor Prefixes

For older browser support:

```
.element {  
  -webkit-animation: fadeIn 1s -webkit-forwards;  
  -moz-animation: fadeIn 1s forwards;  
  animation: fadeIn 1s forwards;  
}
```

Note: `forwards` doesn't need prefix in modern browsers, but be consistent.

Chapter 8

Advanced Usage

8.1 Combining with JavaScript

Control fill-mode dynamically:

```
.element {  
  animation: slideIn 1s ease-out forwards;  
}  
  
.element.temporary {  
  animation-fill-mode: none;  
  /* Reverts to normal state after animation */  
}
```

JavaScript:

```
const element = document.querySelector('.element');  
  
// For permanent effect  
element.classList.add('element');  
  
// For temporary effect  
element.classList.add('element', 'temporary');  
  
// After animation ends  
element.addEventListener('animationend', () => {  
  if (element.classList.contains('temporary')) {  
    element.style.opacity = '0'; // Manually reset if needed  
  }  
});
```

8.2 Multiple Animations with Different Fill-Modes

```
.element {  
  animation:  
    fadeIn 1s ease-in forwards,  
    slideIn 1s ease-out forwards,  
    scaleUp 1s ease-out forwards;  
  /* All three animations play simultaneously */  
}
```

```
/* All stay at their final states */  
}
```

Chapter 9

Quick Reference

9.1 Fill-Mode Values Summary

Value	Before Anim	After Anim	Use Case
none	normal	normal	Repeating
forwards	normal	100%	Stay final state
backwards	0%	normal	Preview start
both	0%	100%	Full control

9.2 Complete Checklist

Is animation one-time or repeating?

Should element stay in final state?

Does animation have a delay?

Should starting state be visible during delay?

Choose appropriate fill-mode value

Test on multiple browsers

Check on mobile devices

9.3 Common Patterns

- **Page load:** forwards (fade in and stay)
- **Spinners:** none (infinite loop)
- **Hover:** none (return on leave)
- **Modal:** forwards (appear and stay)
- **Staggered:** both (preview and stay)
- **Pulse/Bounce:** none (reset between cycles)

9.4 Related Animation Properties

- `animation-name` - Which keyframes to use
- `animation-duration` - How long animation takes
- `animation-timing-function` - Speed curve
- `animation-delay` - Delay before starting
- `animation-iteration-count` - Times to repeat
- `animation-direction` - Forward/reverse/alternate
- `animation-play-state` - Play or pause

9.5 Pro Tips

1. **Default to forwards** for most modern animations
2. Use **none** only for infinite/repeating animations
3. Use **both** when you need both pre-animation state visibility
4. **Test fill-mode** - It's easy to forget and breaks animations
5. **Combine with delays** for complex staggered effects
6. **Document your choice** - Comment why you picked a fill-mode